Three Roads: Choose Your Stack with AI

Track: General AI Fluency
Week: 4
Phase: Build
Workload: 2 hours

1. My Four Constraints

Budget

I want to use free tools and free hosting for my portfolio. I do not want the portfolio to depend on paid services.

My Skill Level

I am comfortable with React, Next.js, JavaScript, TypeScript, GitHub, and basic deployment. I can work with a modern frontend stack, but I want to avoid unnecessary backend and infrastructure complexity that would make the project difficult to maintain.

What My Portfolio Needs to Do

My portfolio needs to include:

1. Home page
2. Work and project pages
3. Detailed case studies
4. About page
5. Contact page
6. Project screenshots
7. Image galleries
8. Long-form case-study content
9. GitHub repository links
10. Live demo links where available
11. Clear calls to action

The portfolio should make my actual work easy to understand and explore.

How My Work Needs to Be Displayed

My work needs to be presented through real screenshots, project images, detailed case studies, GitHub repositories, and live demonstrations where available. The website should also make it easy to add new projects and case studies later.

Backend Requirement

I do not need a backend at this stage. The portfolio mainly needs to present content, images, projects, repositories, and links. A backend can be considered later if a future feature actually requires one.

2. Option 1 — Simplest

Stack

HTML, CSS, and JavaScript

How I Would Build It

I would create a mostly static website using HTML, CSS, and JavaScript. Each portfolio page would be created manually and the project content would be stored directly in the website files.

Free Hosting

GitHub Pages

Backend

No backend required.

Advantages

This would be the simplest option to understand and maintain. It would also be inexpensive because the tools and hosting can be free.

Trade-off

As the portfolio grows, manually managing multiple case studies, project pages, images, and repeated layouts could become inconvenient. It would also give me less structure for building a larger portfolio.

3. Option 2 — Next.js

Stack

Next.js, React, and TypeScript

How I Would Build It

I would create reusable components and pages for the portfolio. Projects and case studies could follow a consistent structure while still having their own content and images.

Free Hosting

Vercel

Backend

No backend required for the current portfolio.

Advantages

Next.js provides more flexibility than a basic static website while still allowing the portfolio to remain relatively simple. It is suitable for project pages, case studies, images, GitHub links, live demos, and future additions.

Trade-off

It requires more setup and knowledge than a basic HTML/CSS/JavaScript website. I would also need to maintain the React and Next.js project structure and dependencies.

4. Option 3 — Full-Stack Next.js Application

Stack

Next.js, React, TypeScript, database, and backend/API services

How I Would Build It

I would build the portfolio as a full-stack application with a backend and database. Project content could potentially be managed dynamically through a database or content management system.

Free Hosting

A combination of free-tier services such as Vercel and a free database service.

Backend

Yes.

Advantages

This would provide the greatest flexibility for future features such as dynamic content management, user interactions, dashboards, or a personal AI agent.

Trade-off

It would introduce significantly more development and maintenance. I would have to manage backend logic, database configuration, authentication or security where required, and additional services.

5. Pressure Test

Simplest Option

The simplest option would be easy to maintain, but managing a growing portfolio manually could become inconvenient. Adding many case studies and reusable layouts would require more manual work.

Next.js Option

Next.js provides enough flexibility for my current needs without requiring a backend. It can handle case studies, project pages, images, GitHub links, and live demos while remaining manageable for me.

Full-Stack Option

The full-stack option would provide more possibilities, but most of those features are not necessary for the current portfolio. The additional backend and database would create more things to maintain.

Two-Week Feasibility

The simplest option could be completed quickly, but Next.js is also realistic because I already have experience with it and have already created the initial portfolio project.

The full-stack option would create unnecessary complexity and make completing the portfolio within two weeks more difficult.

Displaying My Work

All three options can display basic project work, but Next.js gives me a good balance between flexibility and simplicity. It can support detailed case studies, screenshots, galleries, GitHub repositories, and live demos without requiring a backend.

6. My Decision

I chose Next.js with React and TypeScript, hosted on Vercel.

I chose this stack because it matches the way I need to present my work while remaining manageable for me. I can create reusable layouts for projects and case studies, display screenshots and images, link to GitHub repositories and live demos, and expand the portfolio later without rebuilding the entire website.

I also already have experience working with this stack, which makes it more realistic for me to finish and maintain the portfolio.

7. Why I Rejected the Simplest Option

I rejected the basic HTML, CSS, and JavaScript approach because although it would be easy to start and host, managing a growing portfolio with multiple case studies and reusable project layouts would become more manual.

I want the portfolio to be more than a small static page, so I prefer the additional structure provided by Next.js.

8. Why I Rejected the Most Powerful Option

I rejected the full-stack approach because I do not currently need a backend or database for the portfolio.

Adding those technologies would increase development and maintenance requirements without solving an immediate problem. I can add backend functionality later if a specific portfolio feature requires it.

9. Can I Maintain This?

Yes.

I already have experience with React, Next.js, TypeScript, GitHub, and basic deployment. The current portfolio does not require a backend, database, or complex infrastructure, so I can maintain the project myself.

10. Does It Show My Work Well?

Yes.

Next.js can support the way I need to present my work. I can create detailed case studies, display real project screenshots and galleries, link to GitHub repositories, include live demos, and organize multiple projects consistently.

11. Backend Decision

I do not need a backend right now.

The current purpose of the website is to present my work and provide clear ways for visitors to explore projects and contact me. Static and frontend functionality is enough for the current version.

A backend can be considered later if I add features that genuinely require one.

12.Final Stack

Chosen stack:

- Next.js
- React
- TypeScript
- Tailwind CSS where useful
- GitHub
- Vercel

Backend:

Not required at this stage.

Reason for final decision:

I chose this stack because it gives me enough flexibility to present my work professionally while keeping the project within my current skill level and maintenance ability. It is free to start, supports the content and project structure I need, and is realistic to complete within the available time.